Functions

C – Functions

- A function is a block of code that performs a specific task.
- Every C program has **at least one function**, which is **main()**, and all the most trivial programs can define additional functions.
- When the algorithm of a certain problem involves long and complex logic, it is broken into **smaller, independent and reusable blocks**. These small blocks of code are known by different names in different programming languages such as a **module**, a **subroutine**, a **function** or a **method**.
- You can divide up your code into separate functions. How you divide up your code among different functions is up to you, but **logically the division is such that each function performs a specific task.**
- The function contains the **set of programming statements enclosed by {}.**
- A function can be **called multiple times** to provide reusability and modularity to the C program.
- In other words, we can say that the collection of functions creates a program.

C – Functions

Advantage of functions in C

- By using functions, we can avoid rewriting same logic/code again and again in a program. Reusable codes that can be used in other programs. Reusability is the main achievement of C functions.
- The program will be easier to understand, maintain and debug.
- A large program can be divided into smaller modules. Hence, a large project can be divided among many programmers.
- We can call C functions any number of times in a program and from any place in a program.
- We can track a large C program easily when it is divided into multiple functions.
- However, Function calling is always a overhead in a C program.

C – Functions

Types of function

There are two types of function in C programming:

- Standard library functions
- User-defined functions

Standard library functions

- The standard library functions are built-in functions in C programming.
- These functions are defined in header files.

Example,

- The printf() - send formatted output to the screen (display output on the screen) - stdio.h
  The sqrt() - calculates the square root of a number -math.h

C – Functions

Library Function

A library function is also referred to as a **“built-in function”**. A compiler package already exists that contains these functions, each of which has a specific meaning and is included in the package. Built-in functions have the advantage of being directly usable without being defined, whereas user-defined functions must be declared and defined before being used.

For Example:

Pow(), sqrt(), strcmp(), strcpy() etc.

User-defined functions

Create functions as per your requirement.

Such functions created by the user for a specific task are known as user-defined functions.

C – Functions

```
#include <stdio.h>
void functionName()
{
    ... .. ...
    ... .. ...
}

int main()
{
    ... .. ...
    ... .. ...

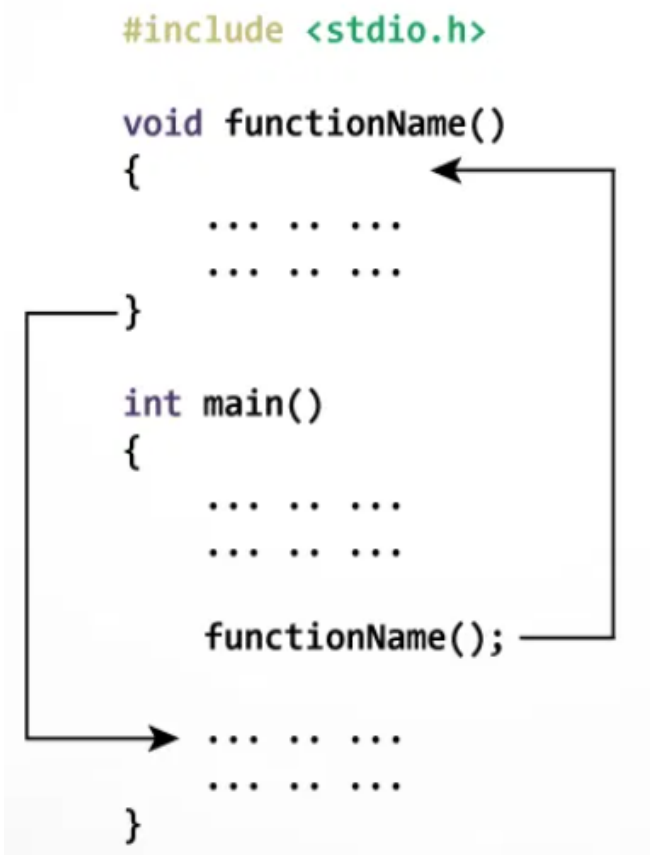
    functionName();

    ... .. ...
    ... .. ...
}
```

Working of user-defined functions

- The execution of a C program begins from the main() function.
- When the compiler encounters functionName();, control of the program jumps to void functionName()
- And, the compiler starts executing the codes inside functionName().
- The control of the program jumps back to the main() function once code inside the function definition is executed.

C – Functions



C – Functions Declaration

A function has 3 parts:

- **Function Declaration/ Function Prototype**
- **Function Definition**
- **Function Calls**

The function_name and parameters list are together known as the **signature** of a function in C programming.

A function **declaration** tells the compiler about a function's name, sequence, return type, and the number and type of its parameters. A function may or may not require parameters. Hence, the parameter list is optional.

A function declaration tells the compiler that there is a function with the given name defined somewhere else in the program.

Syntax

return_type **name_of_the_function** (datatype *parameter_1*, datatype *parameter_2, …*);

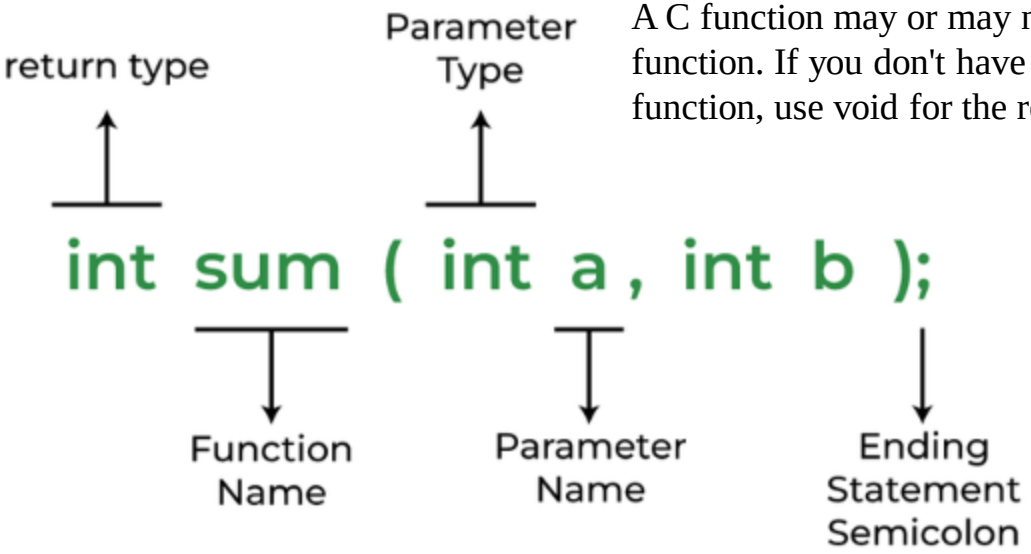
- The parameter name is not mandatory while declaring functions. We can also declare the function without using the name of the data variables.

C – Functions

Example

int **sum**(int *a*, int *b*); // Function declaration with parameter names
int **sum**(int , int); // Function declaration without parameter names

Note: *A function in C must always be declared globally before calling it.*



A C function may or may not return a value from the function. If you don't have to return any value from the function, use void for the return type.

C – Functions definition

A function **definition** provides the actual body of the function.

The function definition consists of actual statements which are executed when the function is called (i.e. when the program control comes to the function). ***Function definition must return only one value at the end of the execution.***

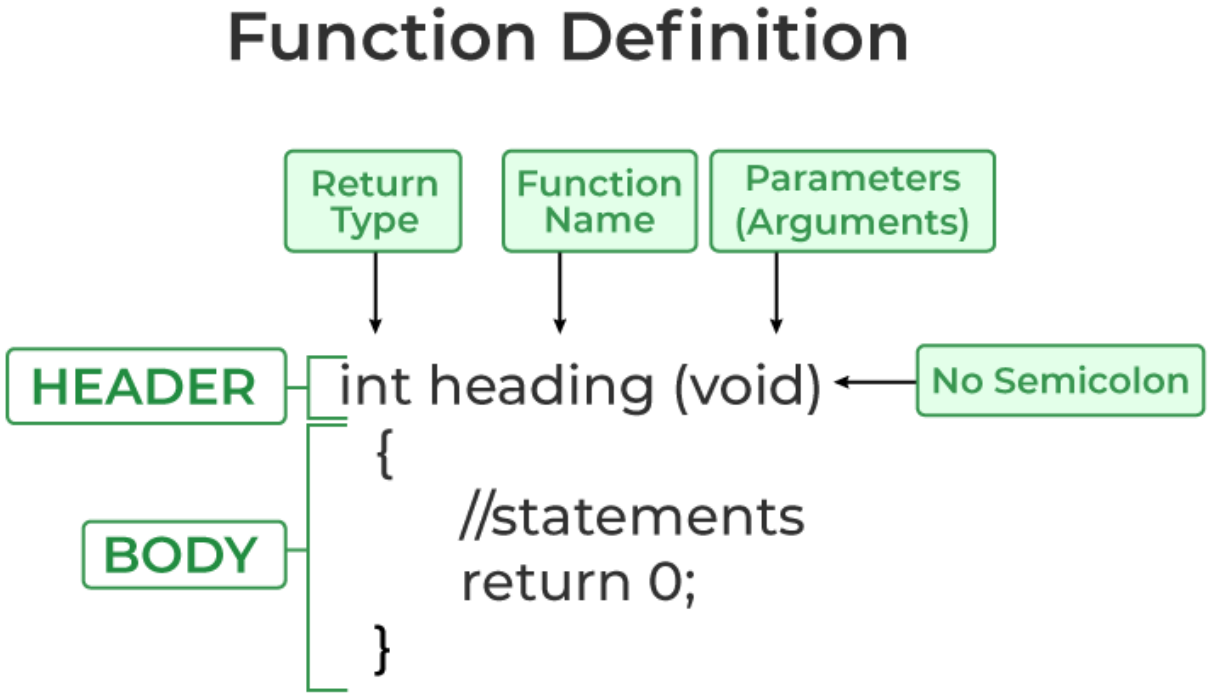
- A C function is generally defined and declared in a single step because the function definition always starts with the function declaration so we do not need to declare it explicitly. The below example serves as both a function definition and a declaration.

The parameter list must not differ in function calling and function declaration. We must pass the same number of functions as it is declared in the function declaration.

return_type **function_name** (para1_type *para1_name*, para2_type *para2_name*)
{
// body of the function
}

Note: *Function call is necessary to bring the program control to the function definition. If not called, the function statements will not be executed.*

C – Functions Definition



C – Functions Call

Function Call

- A function call is a statement that instructs the compiler to execute the function. We use the function name and parameters in the function call.

Note: *Function call is neccessary to bring the program control to the function definition. If not called, the function statements will not be executed*.

Calling a Function in C

- While creating a C function, you give a definition of what the function has to do. To use a function, you will have to call that function to perform the defined task.
- **To call a function properly, you need to comply with the declaration of the function prototype. If the function is defined to receive a set of arguments, the same number and type of arguments must be passed.**
- When a function is defined with arguments, the arguments in front of the function name are called **formal arguments**. When a function is called, the arguments passed to it are the **actual arguments**.
- When a program calls a function, the program control is transferred to the called function. A called function performs a defined task and when its return statement is executed or when its function-ending closing brace is reached, it returns the program control back to the main program.

C – Functions Example

```
#include <stdio.h>
int max(int num1, int num2); /* Function declaration */
int main ()
{
   int a = 100; int b = 200; int ret; /* local variable definition */
   ret = max(a, b);                    /* Function call*/
   printf( "Max value is : %d\n", ret );
   return 0;
}
int max(int num1, int num2) /* Function definition*/
{
int result;                          /* local variable declaration */
if (num1 > num2)
            result = num1;
else
             result = num2;
return result;
}
```

C – Functions

Different aspects of function calling

A function may or may not accept any argument. It may or may not return any value.

Based on these facts, There are four different aspects of function calls.

- Function without arguments and without return value
- Function without arguments and with return value
- Function with arguments and without return value
- Function with arguments and with return value

C – Functions

Function without argument and return value

```
#include<stdio.h>
void printfn();
void main ()
{
   printf("Hello ");
   printfn();
}
void printfn()
{
   printf(“C Programming");
}
```

C – Functions

Function without argument and return value

```
#include<stdio.h>
void sum();
void main()
{
   printf("\nCalculate the sum of two numbers:");
   sum();
}
void sum()
{
   int a,b;
   printf("\nEnter two numbers");
   scanf("%d %d", &a &b);
   printf("The sum is %d",a+b);
}
```

C – Functions

Function without argument and with return value

```
#include<stdio.h>
int sum();
void main()
{
   int result;
   printf("\nCalculate the sum of two numbers:");
   result = sum();
   printf("%d",result);
}
int sum()
{
   int a,b;
   printf("\nEnter two numbers");
   scanf("%d %d",&a,&b);
   return a+b;
}
```

C – Functions

Function without argument and with return value

```
#include<stdio.h>
int square();
void main()
{
   printf(“\nCalculate the area of the square\n");
   float area = square();
   printf("The area of the square: %f\n",area);
}
int square()
{
   float side;
   printf("Enter the length of the side in meters: ");
   scanf("%f", &side);
   return side * side;
}
```

C – Functions

Function with argument and without return value

```
#include<stdio.h>
void sum(int, int);
void main()
{
   int a, b, result;
   printf("\nCalculate the sum of two numbers:");
   printf("\nEnter two numbers:");
   scanf("%d %d",&a,&b);
   sum(a,b);
}
void sum(int a, int b)
{
   printf("\nThe sum is %d", a+b);
}
```

C – Functions

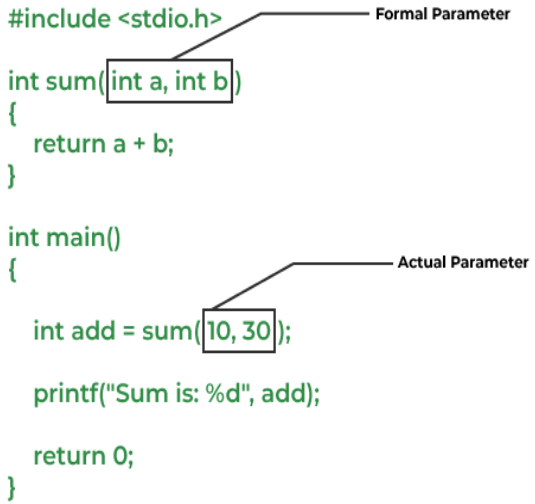
Function with argument and without return value [**program to calculate the average of five numbers.]**

```
#include<stdio.h>
void average(int, int, int, int, int);
void main()
{
   int a,b,c,d,e;
   printf("\nCalculate the average of five numbers:");
   printf("\nEnter five numbers:");
   scanf("%d %d %d %d %d", &a, &b, &c, &d, &e);
   average(a,b,c,d,e);
}
void average(int a, int b, int c, int d, int e)
{
   float avg;
   avg = (a+b+c+d+e)/5;
   printf("The average of given five numbers : %f",avg);
}
```

C – Functions

Function with argument and with return value

```
#include<stdio.h>
int sum(int, int);
void main()
{
   int a,b,result;
   printf("\nGoing to calculate the sum of two numbers:");
   printf("\nEnter two numbers:");
   scanf("%d %d",&a,&b);
   result = sum(a,b);
   printf("\nThe sum is : %d",result);
}
int sum(int a, int b)
{
   return a+b;
}
```



C – Functions

Function with argument and with return value[**Program to check whether a number is even or odd**]

```
#include<stdio.h>
int even_odd(int);
void main()
{
 int n,flag=0;
 printf("\nCheck whether a number is even or odd");
 printf("\nEnter the number: ");
 scanf("%d",&n);
 flag = even_odd(n);
 if(flag == 0)
    printf("\nThe number is odd");
  else
    printf("\nThe number is even");
}
int even_odd(int n)
{
    if(n%2 == 0)
        return 1;
    else
        return 0;
}
```

C – Functions

Advantages of C functions

- **Modular Programming:** The ability to ***divide*** a ***huge program*** into smaller, more manageable modules is one of the main benefits of utilizing functions in C. Each function might contain a particular job or component of functionality, which streamlines and clarifies the overall program structure. This modular strategy improves code reuse and makes maintenance and debugging easier.

- **Code reuse:** By using functions, you may create a certain algorithm or piece of logic only once and utilize it repeatedly throughout your program. You may just call the function anytime you need to run the code, saving you from having to duplicate it elsewhere. It not only ***speeds up development*** but also assures consistency and lessens the possibility of making mistakes.

- **Encapsulation and Abstraction:** By obscuring the specifics of functionality's implementation, functions offer a level of abstraction. A function prototype's interface can be defined in a ***header file***, while the actual implementation can be provided in a different source file. Other portions of the program can utilize the function without needing to understand how it is internally implemented because of the separation of interface and implementation.

C – Functions

Advantages of C functions contd…

- **Easy Program Maintenance:** A program may be understood and maintained more easily if it is divided into smaller functions. The ability to assign a specific responsibility to each function makes code more readable and makes troubleshooting and debugging easier. You may concentrate on the necessary function without impacting other portions of the program if an error is found or a modification is needed.

- **Improved Collaboration:** Functions allow developers working on the same project to collaborate. The program may be divided into functions so that several team members can work on ***different functions*** at once. Developers may smoothly integrate their work into the functions if the interfaces are well specified, which improves productivity and promotes effective development.

C – Functions

Advantages of C functions contd…

- **Passing the parameter:** You can send arguments or data to a function in C so that it can process them. The function can carry out actions and generate outcomes using these inputs. You may increase the flexibility and adaptability of functions by passing parameters, which will increase your program's overall versatility.

- **Return Values:** Functions can send values back to the code that called them, allowing for the communication of the function's execution results. You can utilize the returned value in other areas of the program after doing calculations or data manipulation within the function. ***Return values*** are especially helpful when you need to calculate a result or establish a condition depending on the function's output.

C – Functions [Advantages and Disadvantages in short]

Advantages of Functions in C

- Functions in C is a highly useful feature of C with many advantages as mentioned below:
- The function can reduce the repetition of the same statements in the program.
- The function makes code readable by providing modularity to our program.
- There is no fixed number of calling functions it can be called as many times as you want.
- The function reduces the size of the program.
- Once the function is declared you can just use it without thinking about the internal working of the function.

Disadvantages of Functions in C

- Cannot return multiple values.
- Memory and time overhead due to stack frame allocation and transfer of program control.